Chapter 14: Real-World Case Studies and Patterns

Download this chapter as a PDF for offline reading:
Chapter-14-Real-World-Case-Studies-and-Patterns.pdf

The techniques from earlier chapters become most valuable when they are applied to real vulnerability patterns. This chapter looks at common classes of issues and at the process of turning a single known finding (a seed) into a broader search.

Common Vulnerability Classes

Most security queries fall into a handful of well-known classes. The standard libraries already contain high-quality queries for many of them; understanding the shape of each class helps you write or customize the rest.

Injection

Untrusted data reaches a parser or an interpreter.

- SQL injection — data flows into a query string.
- Command injection — data flows into a shell or process argument.
- Code injection — data flows into `eval`, `Function`, or a template engine.

The pattern is always the same: remote (or otherwise untrusted) source → taint-tracking configuration → sink that interprets the data.

Simplified seed (see `examples/chapter14/sql-injection-seed.ql`):

```
class SqlSeedConfig extends TaintTracking::Configuration {
  SqlSeedConfig() { this = "SqlSeedConfig" }

  override predicate isSource(DataFlow::Node source) {
    source instanceof RemoteFlowSource
  }

  override predicate isSink(DataFlow::Node sink) {
    sink = any(CallExpr c | c.getCalleeName().matches("%query%") or
                            c.getCalleeName().matches("%execute
%")).getArgument(0).flow()
  }
}
```

Cross-site scripting (XSS)

Untrusted data reaches a browser DOM sink (`document.write`, `innerHTML`, etc.). The configuration is almost identical; only the sinks change. See `examples/chapter14/xss-seed.ql`.

Path traversal, SSRF, open redirects

Again the same shape: a source of untrusted data, a taint configuration, and a sink that uses the data as a path, URL, or redirect target. Barriers are usually sanitizers or allow-lists.

In every case the modeling work from Chapters 9 and 10 (sources, sinks, barriers, and summaries) is what makes the query precise.

From Seed to Variant Analysis

A seed is a concrete example of a vulnerability you already understand. The goal is to generalize it.

The process

1. Start with a known vulnerable code snippet (or a result from an existing query).
2. Write the narrowest query that finds that snippet.
3. Relax the source, the sink, or both until the query also finds similar instances.
4. Add barriers and models to remove false positives.
5. Run the refined query across many repositories (MRVA, Chapter 12) to discover variants.

Example

Suppose you know that this line is vulnerable:

```
db.query("SELECT * FROM users WHERE name = '" + req.query.name + "'");
```

The seed query above finds it. Generalizing might mean:

- Accepting any remote source, not only `req.query`.
- Accepting any method whose name contains `query` or `execute`.
- Adding a barrier for a known escaping helper.

After the generalization you run the query with MRVA. The results that look like true positives become candidates for a permanent query; the false positives tell you which additional barriers or models you still need.

This seed-to-variant loop is how most production CodeQL queries are born.

Exercises

1. Take the SQL injection seed and add a barrier for a function named `escapeSql`.
2. Generalize the XSS seed so it also treats assignments to `innerHTML` as sinks.
3. Run one of the seeds with MRVA (or against several local databases) and note which results look like true positives.
4. (Optional) Turn one true-positive path into a more precise query by tightening the source or adding a missing model.

Solutions and additional examples live in the companion repository under `examples/chapter14/`.

Next Steps

You can now recognize the common vulnerability shapes and turn a single known issue into a search that finds its cousins. The final chapter collects the habits that keep this work effective and points to further resources.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter14/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL